



Aprende a criar o teu avatar 3D para a web

Um jogo com Babylon.js, direto no browser

Verão Ciência no IPB · CEDRI
13–17 de julho de 2026

Todo o material desta sessão está aqui:

**[github.com/HelderPereiraIPB/
verao-ciencia-2026-assets](https://github.com/HelderPereiraIPB/verao-ciencia-2026-assets)**

Guarda este link já — vais precisar dele durante a sessão

Antes do 3D no browser

- Até 2011, não havia 3D nativo num browser — só com plugins (Flash, Java, VRML)
- Os utilizadores tinham de instalar esse plugin só para ver algo em 3D

2011 — chega o WebGL

- Mozilla, Google, Apple e Opera juntam-se para criar o WebGL: 3D acelerado pela placa gráfica, direto no browser, sem instalar nada
- developer.mozilla.org/.../WebGL_API

2017 e depois — WebGL2 e WebGPU

- **WebGL2** (2017): mais funcionalidades das placas gráficas mais recentes, sem mudar a base
- developer.mozilla.org/.../WebGL2RenderingContext
- **WebGPU**: a geração seguinte — usa várias threads e placas gráficas modernas, até 3–8× mais rápido em cenas complexas
- developer.mozilla.org/.../WebGPU_API

O que é um motor 3D?

- Um programa que sabe desenhar objetos, luzes e câmaras num ecrã
- Trata da matemática toda por nós (nós só dizemos *o quê* e *onde*)
- Usa-se para jogos, aplicações, e até montras de produtos online

Biblioteca vs. motor

- **Biblioteca** = uma caixa de ferramentas soltas — dá-te o essencial (desenhar formas, câmaras, luzes) e tu escolhes e ligas o resto
- **Motor** = um pacote completo — já vem com física, colisões, som e interface prontos a usar

Three.js vs. Babylon.js

- **Three.js** é uma *biblioteca*: enorme comunidade, muito flexível — mas colisões, física e interface não vêm incluídas, tens de escolher e ligar outras bibliotecas
- **Babylon.js** é um *motor*: colisões, física, interface e carregar modelos .glb já vêm prontos — comesças a fazer jogo mais depressa
- Também existem **PlayCanvas**, **A-Frame** e **React Three Fiber**, entre outras

Three.js vs. Babylon.js — na prática

- Mesmo resultado nos dois: caixa a rodar, luz, câmara com controlo de rato
- **Babylon.js** — 17 linhas de código, um único `<script>`
- **Three.js** — 28 linhas de código, mais um bloco `importmap` e o add-on `OrbitControls` à parte (não vem no núcleo)
- `OrbitControls` é o que dá o controlo do rato — no Babylon.js isso já vem de série na câmara, sem ser preciso ligar mais nada
- Ver e comparar o código-fonte das duas (com explicação do `importmap` e do `OrbitControls`): helderpereiraipb.github.io/verao-ciencia-2026-assets

Ferramentas do Babylon.js

- **Playground** — onde vamos escrever o código hoje
- **Sandbox** — para veres qualquer modelo 3D (.glb) sozinho
- **GUI Editor** — para desenhar texto e botões no ecrã
- **Node Material Editor** — para criar materiais e texturas

Onde arranjar modelos e texturas grátis

- Os modelos de hoje foram feitos no **Blender** e exportados para .glb — um formato universal, aberto a qualquer motor 3D
- **Kenney.nl** — muitos modelos *low-poly* prontos para jogos, o estilo usado hoje
- **Quaternius** — mais modelos *low-poly* (personagens, cenários, objetos)
- **Poly Haven** — texturas e HDRIs realistas
- **ambientCG** — mais texturas realistas
- **Mixamo** — animações prontas para personagens humanas
- Todos gratuitos e sem direitos de autor (CC0) — podem usar à vontade

Vamos programar!

Bloco 1 — Cena, câmara, luz, chão

- Ao correr o código, aparece um fundo azul-claro, um chão castanho e uma neblina suave ao fundo
- É a partir deste ficheiro que o jogo todo vai crescer, bloco a bloco — nada se deita fora

Bloco 1 — código

jogo.js

```
var ESCALA = 7; // personalizavel -- tenta 3 ou 15
var createScene = async function () {
  var scene = new BABYLON.Scene(engine);
  scene.clearColor = new BABYLON.Color4(0.55, 0.8, 0.95, 1); // cor do céu
  scene.fogMode = BABYLON.Scene.FOGMODE_LINEAR; // nevoeiro = profundidade
  scene.fogColor = new BABYLON.Color3(0.75, 0.88, 0.96);
  scene.fogStart = 18; scene.fogEnd = 65;
  var camera = new BABYLON.ArcRotateCamera(
    "cam", -Math.PI / 2, Math.PI / 2.15, 13, new BABYLON.Vector3(4, 1.8, 0), scene
  );
  var light = new BABYLON.HemisphericLight("luz", new BABYLON.Vector3(0, 1, 0), scene);
  light.intensity = 0.9;
  var ceu = BABYLON.MeshBuilder.CreatePlane("ceu", { width: 300, height: 60 }, scene);
  ceu.position.set(0, 15, 20);
  var ceuMat = new BABYLON.StandardMaterial("ceuMat", scene); // ganha textura no bloco 10
  ceuMat.emissiveColor = new BABYLON.Color3(0.4, 0.7, 0.95);
  ceuMat.disableLighting = true; ceuMat.fogEnabled = false; ceu.material = ceuMat;
  var chao = BABYLON.MeshBuilder.CreateGround( // no bloco 10 ganha textura
    "chao", { width: 300, height: 50 }, scene
  );
  var chaoMat = new BABYLON.StandardMaterial("chaoMat", scene);
  chaoMat.diffuseColor = new BABYLON.Color3(0.55, 0.35, 0.2); // tenta (0,1,0)
  chao.material = chaoMat;
  return scene;
};
export default createScene;
```

Sugestões para testar

- Mudar a distância da câmara (13) para 20 ou 5
- Mudar a cor do `chaoMat.diffuseColor` (vermelho, verde, azul, de 0 a 1)
- Mudar o `scene.fogEnd` para 200 e comparar

Bloco 2 — Carregar um modelo 3D

- Aparece o Mallard, de pé e parado, já bem colocado sobre o chão
- Fica virado para a direita, pronto para o próximo bloco (mover-se)

Bloco 2 — código (função de carregar)

jogo.js

```
// no TOPO do ficheiro, ANTES de "var createScene = ...":
var PASTA_MODELOS =
  "https://raw.githubusercontent.com/HelderPereiraIPB/" +
  "verao-ciencia-2026-assets/main/GLB/";
async function carregarModelo(nome, ficheiro, x, y, z, scene, pai) {
  var res = await BABYLON.SceneLoader.ImportMeshAsync("", PASTA_MODELOS, ficheiro, scene);
  var raiz = res.meshes[0];
  raiz.name = nome; // recentra o modelo e poe a base dele em y=0
  var bb = raiz.getHierarchyBoundingVectors();
  var centroX = (bb.min.x + bb.max.x) / 2;
  var centroZ = (bb.min.z + bb.max.z) / 2;
  var baseY = bb.min.y;
  raiz.position.x -= centroX;
  raiz.position.z -= centroZ;
  raiz.position.y -= baseY;
  raiz.scaling.x *= ESCALA;
  raiz.scaling.y *= ESCALA;
  raiz.scaling.z *= ESCALA;
  raiz.position.x = raiz.position.x * ESCALA + x;
  raiz.position.y = raiz.position.y * ESCALA + y;
  raiz.position.z = raiz.position.z * ESCALA + z;
  if (pai) raiz.setParent(pai);
  return { raiz: raiz, animationGroups: res.animationGroups };
}
```

Bloco 2 — código (usar a função)

jogo.js

```
// dentro do createScene, logo a seguir a "chao.material = chaoMat;"  
// -- ANTES do "return scene;" que ja existe no fim do ficheiro  
// (nao o dupliques, so fica lá uma vez):  
var mallardObj = await carregarModelo(  
    "Mallard", "Mallard_Low_Poly.glb", 0, 0, 0, scene  
);  
var mallard = mallardObj.raiz;  
mallard.rotationQuaternion = null; // permite rodar pelo eixo Y  
mallard.rotation.y = -Math.PI / 2; // vira para a direita
```

Bloco 3 — O mundo desliza

- D ou seta direita faz o Mallard andar, com a animação das patas a ligar-se sozinha
- A ou seta esquerda anda para trás; ao soltar a tecla, o Mallard para

Bloco 3 — código (teclas e estado)

jogo.js

```
// dentro do createScene, depois de carregar o Mallard:
var nivel = new BABYLON.TransformNode("nivel", scene);
var VELOCIDADE_ANDAR = 6; // tenta 15, o Mallard fica bem mais rapido

var teclas = {};
window.addEventListener("keydown", function (e) {
    teclas[e.key.toLowerCase()] = true;
});
window.addEventListener("keyup", function (e) {
    teclas[e.key.toLowerCase()] = false;
});

var estado = { aFacingDireita: true };
var walkAnim = mallardObj.animationGroups.find(a => a.name === "Walk");
```

Bloco 3 — código (o ciclo do jogo)

jogo.js

```
// este ciclo corre uma vez por cada frame desenhado -- cola-o dentro
// do createScene, ANTES do "return scene;" que ja existe (nao o
// duplique, so fica lá uma vez, no fim do ficheiro):
scene.onBeforeRenderObservable.add(function () {
    // limite de seguranca: um salto grande no tempo (ex: logo a seguir
    // ao carregamento) nao deve mexer o jogo de repente
    var dt = Math.min(engine.getDeltaTime() / 1000, 0.1);
    var andou = false;
    if (teclas["d"] || teclas["arrowright"]) {
        nivel.position.x -= VELOCIDADE_ANDAR * dt;
        estado.aFacingDireita = true;
        andou = true;
    }
    if (teclas["a"] || teclas["arrowleft"]) {
        nivel.position.x += VELOCIDADE_ANDAR * dt;
        estado.aFacingDireita = false;
        andou = true;
    }
    mallard.rotation.y = estado.aFacingDireita ? -Math.PI / 2 : Math.PI / 2;
    if (andou && walkAnim && !walkAnim.isPlaying) walkAnim.start(true);
    if (!andou && walkAnim && walkAnim.isPlaying) walkAnim.stop();
});
```

Bloco 4 — Saltar

- Espaço, W ou seta para cima faz o Mallard saltar
- Sobe, trava no ar e volta a descer sozinho até tocar no chão outra vez

Bloco 4 — código

jogo.js

```
// a seguir a "var estado = {...}" do bloco 3:
var VELOCIDADE_SALTO = 12; // tenta 20, um salto bem mais alto
var GRAVIDADE = -22; // tenta -10, saltos mais lentos e flutuantes
estado.velocidadeY = 0;
estado.noChao = true;

// dentro do onBeforeRenderObservable, antes do "});" final:
if ((teclas[" "] || teclas["w"] || teclas["arrowup"]) && estado.noChao) {
    estado.velocidadeY = VELOCIDADE_SALTO;
    estado.noChao = false;
}
estado.velocidadeY += GRAVIDADE * dt;
mallard.position.y += estado.velocidadeY * dt;

if (estado.velocidadeY <= 0 && mallard.position.y <= 0) {
    mallard.position.y = 0;
    estado.velocidadeY = 0;
    estado.noChao = true;
}
```

Antes do próximo bloco — o que é um array

- [] = uma lista de itens, um a seguir ao outro
- { } = um “cartão” com informação, cada uma com um nome
- A partir de agora é sempre a mesma receita: uma lista de cartões

exemplo

```
var obstaculosFixos = [  
  { ficheiro: "Pilar_Low_Poly.glb", x: 9, y: 0 },  
  { ficheiro: "Barrel.glb", x: 24, y: 0 },  
];
```

- Já programaste em Python? É uma lista de dicionários

Bloco 5 — Obstáculos que bloqueiam

- Este bloco tem 2 partes de código: a primeira acrescenta o Pilar e os Barris, a segunda liga o bloqueio — sem a segunda, os objetos aparecem mas o Mallard atravessa-os na mesma
- No fim das duas, já não os atravessa — tem de saltar para cima deles
- O Pilar fica bloqueado até ao Bloco 9 (escadas automáticas) — não é erro se ainda não conseguires passar por cima dele

Bloco 5 — código (a lista e o carregamento)

jogo.js

```
// a seguir ao bloco 4, ainda dentro do createScene:
var LARGURA_MALLARD = 1.1;
var MARGEM_BLOQUEIO = 1.2; // tolerancia -- nao exige saltar a altura toda

var obstaculosFixos = [
  { ficheiro: "Pilar_Low_Poly.glb", x: 9, y: 0 },
  { ficheiro: "Barrel.glb", x: 24, y: 0 },
  { ficheiro: "Barrel.glb", x: 54, y: 0 },
]; // tenta mudar os "x" ou trocar o ficheiro

var plataformas = [];
for (var i = 0; i < obstaculosFixos.length; i++) {
  var of = obstaculosFixos[i];
  var objFixo = await carregarModelo(
    "obstaculo" + i, of.ficheiro, of.x, of.y, 0, scene, nivel
  );
  var bbFixo = objFixo.raiz.getHierarchyBoundingVectors();
  plataformas.push({
    x: of.x,
    altura: bbFixo.max.y - of.y,
    raioBloqueio: LARGURA_MALLARD / 2 + (bbFixo.max.x - bbFixo.min.x) / 2,
  });
}
```

Bloco 5 — código (bloquear e pousar em cima)

jogo.js

```
// as colisoes prontas do Babylon assumem um boneco a andar num cenario
// parado -- aqui e o cenario que desliza, por isso comparamos posicoes
// a nossa maneira. Dentro do onBeforeRenderObservable, antes do "if (teclas[d]...)":
var bloqueadoAAvancar = plataformas.some(function (p) {
  var dx = Math.abs(nivel.position.x + p.x);
  return dx < p.raioBloqueio && mallard.position.y < p.altura - MARGEM_BLOQUEIO;
});
// antes de mexer no velocidadeY:
var suporteY = 0;
plataformas.forEach(function (p) {
  var dx = Math.abs(nivel.position.x + p.x);
  var temAltura = mallard.position.y >= p.altura - MARGEM_BLOQUEIO;
  if (dx < p.raioBloqueio && p.altura > suporteY && temAltura) suporteY = p.altura;
});
// a verificacao de aterrar troca o "0" por "suporteY":
if (estado.velocidadeY <= 0 && mallard.position.y <= suporteY) {
  mallard.position.y = suporteY;
  estado.velocidadeY = 0;
  estado.noChao = true;
}
```

IMPORTANTE — isto não se cola, é uma edição: vai ao if de andar para a direita que já tens do bloco 3, e muda-o para:

```
if ((teclas["d"] || teclas["arrowright"]) && !bloqueadoAAvancar) { ... }
```

Bloco 6 — Objetos que dão (ou tiram) pontos

- Aparecem o presunto, a bola e a galinha
- Tocar neles faz os pontos subir ou descer, e o objeto desaparece
- Saltar bem alto por cima passa ao lado, sem tocar em nada

Bloco 6 — código

jogo.js

```
// a seguir ao bloco 5, ainda dentro do createScene, ANTES do
// "return scene;" que já existe:
var RAIIO_TOQUE = 1.1;
var ALTURA_LIMPAR_PERIGO = 1.5; estado.pontos = 0; // saltar alto = passa ao lado
var pontuaveis = [ // pontos negativo = tira pontos! tenta pontos: -20 na galinha
  { ficheiro: "Presunto_Low_Poly.glb", x: 16, y: 1, pontos: 10 },
  { ficheiro: "Basketballv2.glb", x: 32, y: 1, pontos: 5 },
  { ficheiro: "Basketballv2.glb", x: 46, y: 1, pontos: 5 },
  { ficheiro: "Presunto_Low_Poly.glb", x: 64, y: 1, pontos: 10 },
  { ficheiro: "Sprite_Crossy_Road.glb", x: 39, y: 0, pontos: -5 },
];
var objetosPontuaveis = [];
for (var j = 0; j < pontuaveis.length; j++) { var pv = pontuaveis[j];
  var raizPv = (await carregarModelo("pontuavel" + j, pv.ficheiro, pv.x, pv.y, 0, scene, nivel)).raiz;
  objetosPontuaveis.push({ raiz: raizPv, x: pv.x, pontos: pv.pontos, apanhado: false });
}

// dentro do onBeforeRenderObservable, antes do "});" final:
objetosPontuaveis.forEach(function (item) { if (item.apanhado) return;
  var dx = Math.abs(nivel.position.x + item.x);
  var passouPorCima = mallard.position.y > ALTURA_LIMPAR_PERIGO;
  if (dx < RAIIO_TOQUE && !passouPorCima) { item.apanhado = true;
    estado.pontos += item.pontos; // soma ou subtrai, conforme o sinal
    item.raiz.dispose();
  }
});
```

Bloco 7 — Interface: pontos, tempo, mensagens

- Aparecem os pontos (canto superior esquerdo) e o tempo (canto superior direito)
- Ficam sempre visíveis, mesmo quando o jogo termina

Bloco 7 — código (pontos e tempo)

jogo.js

```
// a seguir ao bloco 6, ainda dentro do createScene, ANTES do
// "return scene;" que ja existe:
var TEMPO_JOGO_SEGUNDOS = 60; // tenta 20, um jogo bem mais apertado
estado.tempoRestante = TEMPO_JOGO_SEGUNDOS;
estado.fimDeJogo = false;
var gui = BABYLON.GUI.AdvancedDynamicTexture.CreateFullscreenUI("gui");
var textoPontos = new BABYLON.GUI.TextBlock();
textoPontos.text = "Pontos: 0"; textoPontos.color = "white";
// resizeToFit ajusta a caixa ao texto -- sem isto ocupava o ecrã todo
textoPontos.fontSize = 28; textoPontos.resizeToFit = true;
textoPontos.horizontalAlignment = BABYLON.GUI.Control.HORIZONTAL_ALIGNMENT_LEFT;
textoPontos.verticalAlignment = BABYLON.GUI.Control.VERTICAL_ALIGNMENT_TOP;
textoPontos.left = "20px";
textoPontos.top = "15px";
gui.addControl(textoPontos);
var textoTempo = new BABYLON.GUI.TextBlock();
textoTempo.text = "Tempo: " + TEMPO_JOGO_SEGUNDOS; textoTempo.color = "white";
textoTempo.fontSize = 28; textoTempo.resizeToFit = true;
textoTempo.horizontalAlignment = BABYLON.GUI.Control.HORIZONTAL_ALIGNMENT_RIGHT;
textoTempo.verticalAlignment = BABYLON.GUI.Control.VERTICAL_ALIGNMENT_TOP;
textoTempo.left = "-20px";
textoTempo.top = "15px";
gui.addControl(textoTempo);
```

Bloco 7 — código (mensagem final e contagem)

jogo.js

```
// a seguir a "gui.addControl(textoTempo);" do slide anterior, ainda
// dentro do createScene, ANTES do "return scene;" que já existe:
var textoFim = new BABYLON.GUI.TextBlock();
textoFim.text = "";
textoFim.color = "white";
textoFim.fontSize = 32;
textoFim.horizontalAlignment = BABYLON.GUI.Control.HORIZONTAL_ALIGNMENT_CENTER;
textoFim.verticalAlignment = BABYLON.GUI.Control.VERTICAL_ALIGNMENT_TOP;
textoFim.top = "60px";
textoFim.textWrapping = true;
textoFim.width = "80%";
textoFim.height = "80px";
gui.addControl(textoFim);

// logo no inicio do onBeforeRenderObservable, a seguir ao "var dt = ...":
if (estado.fimDeJogo) return;

estado.tempoRestante -= dt;
if (estado.tempoRestante <= 0) {
    estado.tempoRestante = 0;
    estado.fimDeJogo = true;
    textoFim.text = "Tempo esgotado! Pontos: " + estado.pontos;
    if (walkAnim && walkAnim.isPlaying) walkAnim.stop(); // para o Mallard de "andar" parado
}
textoTempo.text = "Tempo: " + Math.ceil(estado.tempoRestante);
textoPontos.text = "Pontos: " + estado.pontos;
```

Bloco 8 — Meta e vitória

- Aparece uma bandeira amarela no fim do nível
- Tocar nela termina o jogo e mostra a pontuação final
- Chegar com tempo a mais dá pontos extra

Bloco 8 — código (comprimento e bandeira)

jogo.js

```
// a seguir ao bloco 7, ainda dentro do createScene, ANTES do
// "return scene;" que ja existe (nao o duplique):
var RATIO_TOQUE_META = 8; // distancia a que a bandeira conta "alcançada"
var BONUS_POR_SEGUNDO = 2; // tenta 0, so os pontos apanhados contam
var MARGEM_META = 20;
// a bandeira coloca-se sozinha a seguir ao ultimo objeto -- nunca fica
// impossivel so por a meta ficar demasiado perto
var COMPRIMENTO_NIVEL = Math.max(
    0,
    ...obstaculosFixos.map(function (o) { return o.x; }),
    ...pontuaveis.map(function (p) { return p.x; })
) + MARGEM_META;
var meta = BABYLON.MeshBuilder.CreateBox(
    "meta", { width: 0.5, height: 4, depth: 0.5 }, scene
);
meta.position.set(COMPRIMENTO_NIVEL, 2, 0);
meta.parent = nivel;
var metaMat = new BABYLON.StandardMaterial("metaMat", scene);
metaMat.diffuseColor = new BABYLON.Color3(1, 1, 0); // tenta (1,0,1)
meta.material = metaMat;
```

Bloco 8 — código (chegar lá)

jogo.js

```
// dentro do onBeforeRenderObservable, antes do "});" final:
if (!meta.isDisposed()) {
  var dxMeta = Math.abs(nivel.position.x + COMPRIMENTO_NIVEL);
  if (dxMeta < RAIIO_TOQUE_META) {
    estado.fimDeJogo = true;
    estado.pontos += Math.floor(estado.tempoRestante) * BONUS_POR_SEGUNDO;
    textoFim.text = "Chegaste! Pontos finais: " + estado.pontos;
    textoPontos.text = "Pontos: " + estado.pontos; // atualiza o canto também, com o bonus incluído
    if (walkAnim && walkAnim.isPlaying) walkAnim.stop();
  }
}
```

Sugestões para testar

- Mudar o BONUS_POR_SEGUNDO para 10 — chegar depressa passa a valer muito mais
- Mudar a MARGEM_META para 5 — a bandeira fica mais perto do último objeto

Bloco 9 — O jogo desenha as escadas sozinho

- Aparecem degraus sozinhos antes do Pilar
- O Mallard consegue subir aos poucos, em vez de ficar bloqueado

Bloco 9 — código (a fórmula e a função)

jogo.js

```
// ainda na parte de configuracao do createScene (NAO dentro do
// onBeforeRenderObservable) -- a seguir ao bloco 8, mas ANTES do
// array obstaculosFixos do bloco 5:
var ALTURA_MAXIMA_SALTO =
    (VELOCIDADE_SALTO * VELOCIDADE_SALTO) / (2 * Math.abs(GRAVIDADE));
var ALTURA_MAXIMA_SALTO_SEGURA = ALTURA_MAXIMA_SALTO * 0.8;
var ESPACAMENTO_DEGRAUS = 2.5;
var LARGURA_DEGRAU = 2;
var PROFUNDIDADE_DEGRAU = 3;
var degraus = [];
// calcula os degraus que faltam para um obstaculo desta altura ser
// saltavel aos poucos
function gerarDegrausAutomaticos(xObstaculo, alturaObstaculo) {
    var lista = [];
    if (alturaObstaculo - MARGEM_BLOQUEIO <= ALTURA_MAXIMA_SALTO_SEGURA) return lista;
    var numDegraus =
        Math.ceil(alturaObstaculo / ALTURA_MAXIMA_SALTO_SEGURA) - 1;
    for (var k = 1; k <= numDegraus; k++) {
        lista.push({
            x: xObstaculo - ESPACAMENTO_DEGRAUS * (numDegraus - k + 1),
            altura: (alturaObstaculo * k) / (numDegraus + 1),
        });
    }
    return lista;
}
```

Bloco 9 — 1/3: troca dentro do ciclo do bloco 5

No bloco 5, dentro do for (var i = 0; i < obstaculosFixos.length; i++) { ... }, já tens estas linhas:

jogo.js --- procura e apaga estas linhas

```
plataformas.push({  
  x: of.x,  
  altura: bbFixo.max.y - of.y,  
  raioBloqueio: LARGURA_MALLARD / 2 + (bbFixo.max.x - bbFixo.min.x) / 2,  
});
```

Troca-as por estas (ficam no mesmo sítio, dentro do mesmo for):

jogo.js --- põe estas no lugar

```
var alturaFixo = bbFixo.max.y - of.y;  
plataformas.push({  
  x: of.x, altura: alturaFixo, raioBloqueio: LARGURA_MALLARD / 2 + (bbFixo.max.x - bbFixo.min.x) / 2,  
});  
degraus = degraus.concat(gerarDegrausAutomaticos(of.x, alturaFixo));
```

Bloco 9 — 2/3: constrói os degraus

Depois do for do bloco 5 fechar (a seguir ao } que fecha esse ciclo, ainda dentro do createScene), acrescenta este código novo:

jogo.js --- acrescentar a seguir ao "for" do bloco 5

```
var degrauMat = new BABYLON.StandardMaterial("degrauMat", scene);
degrauMat.diffuseColor = new BABYLON.Color3(0.7, 0.6, 0.3); // tenta (0.8,0.8,0.8)
degraus.forEach(function (d) {
    var degrau = BABYLON.MeshBuilder.CreateBox("degrau", {
        width: LARGURA_DEGRAU, height: d.altura, depth: PROFUNDIDADE_DEGRAU
    }, scene);
    degrau.position.set(d.x, d.altura / 2, 0);
    degrau.material = degrauMat; degrau.parent = nivel;
    plataformas.push({ x: d.x, altura: d.altura, raioBloqueio: LARGURA_MALLARD / 2 + LARGURA_DEGRAU / 2 })
});
```

Bloco 9 — 3/3: troca o COMPRIMENTO_NIVEL do bloco 8

Já tens isto do bloco 8:

jogo.js --- procura e apaga estas linhas

```
var COMPRIMENTO_NIVEL = Math.max(  
  0,  
  ...obstaculosFixos.map(function (o) { return o.x; } ),  
  ...pontuaveis.map(function (p) { return p.x; } )  
  ) + MARGEM_META;
```

Troca por esta versão (só ganha 1 linha, o resto é igual):

jogo.js --- põe esta no lugar

```
var COMPRIMENTO_NIVEL = Math.max(  
  0,  
  ...obstaculosFixos.map(function (o) { return o.x; } ),  
  ...pontuaveis.map(function (p) { return p.x; } ),  
  ...degraus.map(function (d) { return d.x; } )  
  ) + MARGEM_META;
```

Bloco 10 — Melhorias rápidas

- (a) sem mudança visível — só fica mais rápido a carregar
- (b) o céu passa de cor lisa a uma textura desenhada
- (c) a câmara desloca-se suavemente no fim, em vez de saltar de repente

Bloco 10a — cache de modelos repetidos

jogo.js

```
// no TOPO do ficheiro, ANTES de "var createScene = ...": se dois
// obstaculos usarem o mesmo ficheiro (ex: os dois barris), para que
// descarregar o mesmo .glb da internet duas vezes? guarda-se o
// primeiro numa "cache" e clona-se nas vezes seguintes
var cacheModelos = {};
async function carregarModeloRapido(nome, ficheiro, x, y, z, scene, pai) {
  if (!cacheModelos[ficheiro]) {
    var base = (await carregarModelo("base", ficheiro, 0, 0, 0, scene)).raiz;
    base.setEnabled(false); // e so um molde, nunca aparece sozinho
    cacheModelos[ficheiro] = base;
  }
  var raiz = cacheModelos[ficheiro].clone(nome, null);
  raiz.setEnabled(true);
  raiz.position.set(x, y, z);
  if (pai) raiz.setParent(pai);
  return raiz;
}
// procura as 2 chamadas a "carregarModelo(" que ja tens -- uma no
// ciclo dos obstaculos (bloco 5), outra no ciclo dos pontuaveis
// (bloco 6) -- e troca so o nome da funcao para "carregarModeloRapido("
```

Bloco 10b — texturas desenhadas por código

jogo.js

```
// no TOPO do ficheiro, ANTES de "var createScene = ...": uma textura
// nao precisa de vir de uma imagem -- pode ser desenhada num canvas,
// por codigo (aqui: um gradiente vertical simples)
function criarTexturaGradiente(scene, corTopo, corBase) {
    var tex = new BABYLON.DynamicTexture(
        "grad", { width: 4, height: 256 }, scene, false
    );
    var ctx = tex.getContext();
    var grad = ctx.createLinearGradient(0, 0, 0, 256);
    grad.addColorStop(0, corTopo);
    grad.addColorStop(1, corBase);
    ctx.fillStyle = grad;
    ctx.fillRect(0, 0, 4, 256);
    tex.update();
    return tex;
}
// no bloco 1, tira esta linha do ceuMat:
//   ceuMat.emissiveColor = new BABYLON.Color3(0.4, 0.7, 0.95);
// e poe esta no lugar:
ceuMat.emissiveTexture = criarTexturaGradiente(scene, "#4f95d6", "#cfe9f8");
// no ficheiro final ha tambem versoes para as colinas (silhueta
// ondulada) e para o chao (manchas de textura) -- mesmo principio
```

Bloco 10c — câmara suave ao chegar à meta

jogo.js

```
// cola isto logo a seguir a linha "if (walkAnim && walkAnim.isPlaying)
// walkAnim.stop();" que ja tens do bloco 8 -- ainda dentro do mesmo
// "if (dxMeta < RAI0_TOQUE_META) { ... }", antes do "}" que o fecha.
// A camara desloca-se aos poucos (em vez de saltar de repente) para o
// boneco tambem sair da posicao fixa que tem sempre:
BABYLON.Animation.CreateAndStartAnimation(
    "panCamaraFim", camera, "target.x", 30, 20,
    camera.target.x, camera.target.x - 5,
    BABYLON.Animation.ANIMATIONLOOPMODE_CONSTANT, new BABYLON.CubicEase()
);
```

Agora é convosco!

Segue as missões abaixo, uma de cada vez — cada uma diz exatamente o que mudar, onde mudar, e o que vais ver

Dica: usa Ctrl+F (ou Cmd+F, no Mac) dentro do Playground para encontrar rapidamente qualquer nome de variável ou array no teu código

Missão 1a — acrescentar um obstáculo

jogo.js

```
// no bloco 5, o array obstaculosFixos ja existe no teu codigo com 3
// obstaculos. Acrescenta a ultima linha (marcada "nova"), logo antes
// do "];" que fecha o array -- o array fica assim:
var obstaculosFixos = [
  { ficheiro: "Pilar_Low_Poly.glb", x: 9, y: 0 },
  { ficheiro: "Barrel.glb", x: 24, y: 0 },
  { ficheiro: "Barrel.glb", x: 54, y: 0 },
  { ficheiro: "Barrel.glb", x: 70, y: 0 }, // <-- linha nova
];
```

- O que vais ver: aparece mais um barril, mais à frente no nível (x: 70, depois do último barril em x: 54)
- Se este barril for alto demais para saltar de uma vez, o bloco 9 cria sozinho os degraus que faltam — não precisas de fazer nada extra

Missão 1b — acrescentar um objeto que dá pontos

jogo.js

```
// no bloco 6, o array pontuaveis ja existe no teu codigo com 5
// objetos. Acrescenta a ultima linha (marcada "nova"), logo antes
// do "]" que fecha o array -- o array fica assim:
var pontuaveis = [
  { ficheiro: "Presunto_Low_Poly.glb", x: 16, y: 1, pontos: 10 },
  { ficheiro: "Basketballv2.glb", x: 32, y: 1, pontos: 5 },
  { ficheiro: "Basketballv2.glb", x: 46, y: 1, pontos: 5 },
  { ficheiro: "Presunto_Low_Poly.glb", x: 64, y: 1, pontos: 10 },
  { ficheiro: "Sprite_Crossy_Road.glb", x: 39, y: 0, pontos: -5 },
  { ficheiro: "Presunto_Low_Poly.glb", x: 76, y: 1, pontos: 10 }, // <-- linha nova
];
```

- O que vais ver: aparece mais um presunto perto do fim (x: 76); tocar nele soma 10 pontos, tal como o outro presunto

Missão 2 — muda a dificuldade

jogo.js

```
// no bloco 3, a variavel VELOCIDADE_ANDAR ja existe no teu codigo  
// com o valor 6. Procura essa linha e troca so o numero:  
var VELOCIDADE_ANDAR = 10; // era 6, agora o Mallard anda mais depressa  
  
// no bloco 7, a variavel TEMPO_JOGO_SEGUNDOS ja existe no teu codigo  
// com o valor 60. Procura essa linha e troca so o numero:  
var TEMPO_JOGO_SEGUNDOS = 30; // era 60, agora ha bem menos tempo
```

- O que vais ver: o Mallard anda mais depressa, mas há bem menos tempo para chegar à meta — o jogo fica mais difícil de terminar

Missão 3 — muda o visual

jogo.js

```
// no bloco 1, a linha chaoMat.diffuseColor ja existe no teu codigo.  
// Procura essa linha e troca so a cor (valores de 0 a 1: vermelho,  
// verde, azul):  
chaoMat.diffuseColor = new BABYLON.Color3(0.1, 0.35, 0.1); // chao verde escuro  
  
// no bloco 8, a linha metaMat.diffuseColor ja existe no teu codigo.  
// Procura essa linha e troca so a cor:  
metaMat.diffuseColor = new BABYLON.Color3(1, 0, 1); // bandeira roxa
```

- O que vais ver: o cenário fica com outro aspeto, sem mudar em nada a jogabilidade — boa missão para experimentar várias cores seguidas

Extra — texturas e céus a sério

- As duas missões seguintes trocam as cores lisas por ficheiros reais, gratuitos e sem direitos de autor (CC0), tirados de sites que já vimos antes
- São opcionais — só para quem já terminou as missões 1–3 e quer ir mais longe
- Os ficheiros já estão prontos a usar num link nosso — não precisas de descarregar nem de criar conta em lado nenhum

Extra — textura PBR a sério (1/2): tira do bloco 1

jogo.js --- remover estas 2 linhas

```
var chaoMat = new BABYLON.StandardMaterial("chaoMat", scene);  
chaoMat.diffuseColor = new BABYLON.Color3(0.55, 0.35, 0.2);
```

- A linha `chao.material = chaoMat;`, logo a seguir no bloco 1, não muda — mantém-se igual
- A seguir: o que pôr no lugar destas 2 linhas

Extra — textura PBR a sério (2/2): põe no lugar

jogo.js --- acrescentar no lugar

```
var chaoMat = new BABYLON.PBRMaterial("chaoMat", scene); // agora e PBR
var PASTA_TEXTURAS =
  "https://raw.githubusercontent.com/HelderPereiraIPB/" +
  "verao-ciencia-2026-assets/main/Texturas/";
var texCor = new BABYLON.Texture(PASTA_TEXTURAS + "relva_cor.jpg", scene);
texCor.uScale = 6; texCor.vScale = 6; // repete a imagem 6x6 vezes pelo chao
chaoMat.albedoTexture = texCor; // a cor/imagem em si (ambientcg.com/view?id=Grass004)
var texNormal = new BABYLON.Texture(PASTA_TEXTURAS + "relva_normal.jpg", scene);
texNormal.uScale = 6; texNormal.vScale = 6;
chaoMat.bumpTexture = texNormal; // da relevo (sombras), sem mexer na geometria
var texRugosidade = new BABYLON.Texture(PASTA_TEXTURAS + "relva_rugosidade.jpg", scene);
texRugosidade.uScale = 6; texRugosidade.vScale = 6;
chaoMat.metallicTexture = texRugosidade; // rugosidade real (ambientCG)
chaoMat.useRoughnessFromMetallicTextureGreen = true; // le a rugosidade desta imagem
chaoMat.metallic = 0; // a relva nao e metal
```

Extra — céu real com HDRI (Poly Haven)

```
jogo.js --- acrescentar a seguir a "var light = ..."
```

```
var PASTA_TEXTURAS =  
  "https://raw.githubusercontent.com/HelderPereiraIPB/" +  
  "verao-ciencia-2026-assets/main/Texturas/";  
var hdrCeu = new BABYLON.HDRCubeTexture(  
  // céu real de 360 graus (polyhaven.com/a/kloofendal_48d_partly_cloudy_puresky)  
  PASTA_TEXTURAS + "ceu_polyhaven.hdr", scene, 256  
);  
scene.createDefaultSkybox(hdrCeu, false, 1000); // cria e coloca o céu sozinho
```

- Não precisas de apagar nada — o céu novo cobre sozinho o `scene.clearColor`, que pode ficar como está

Mais ideias, se sobrar tempo

- ESCALA — muda o tamanho de tudo (Mallard incluído); as escadas ajustam-se sozinhas
- degraus — aceita também obstáculos simples à mão, sem modelo nenhum
- A câmara — distância (13) e para onde olha (`new BABYLON.Vector3(4, 1.8, 0)`)
- Um modelo .glb próprio, se já o tiveres pronto e acessível por link

Extra — Modelos próprios e escalas específicas

- Vimos como usar modelos vindos do GitHub do professor, mas e se descarregares um da internet?
- Podes criar uma pasta `modelos` local e colocar lá os ficheiros.
- Como os modelos novos podem ser gigantes, vamos também ensinar o código a aceitar uma escala individual.

Extra — VS Code

- No teu ambiente de trabalho cria uma nova pasta (ex: jogo 3d)
- Abre o VS Code no teu computador e seleciona a pasta que criaste
- Baixa este ficheiro **HTML**
- E copia o teu código do playground para um ficheiro `babylon final.ts` no teu projeto

Extra — A estrutura da tua pasta

- Para que o jogo funcione bem no navegador, a organização dos ficheiros é muito importante.
- Depois de transferires o HTML e criares o ficheiro de código, a tua pasta deve ficar exatamente com este aspeto:

Árvore de ficheiros

```
jogo 3d/  
├─ index.html  
└─ babylon final.ts
```

- O ficheiro HTML que baixaste e o ficheiro TypeScript que criaste têm de estar lado a lado, dentro da mesma pasta.

Extra — código (suporte a ficheiros locais e escalas)

babylon final.ts

```
// no TOPO do ficheiro, altera a funcao carregarModelo  
// para verificar ficheiros com "." e aceitar a "escalaExtra":  
async function carregarModelo(nome, ficheiro, x, y, z, scene, pai, escalaExtra) {  
    var basePath = ficheiro.startsWith(".") ? "" : PASTA_MODELOS;  
    var res = await BABYLON.SceneLoader.ImportMeshAsync("", basePath, ficheiro, scene);  
    var raiz = res.meshes[0];  
    raiz.name = nome; // recentra o modelo e poe a base dele em y=0  
    var bb = raiz.getHierarchyBoundingVectors();  
    var centroX = (bb.min.x + bb.max.x) / 2;  
    var centroZ = (bb.min.z + bb.max.z) / 2;  
    var baseY = bb.min.y;  
    raiz.position.x -= centroX;  
    raiz.position.z -= centroZ;  
    raiz.position.y -= baseY;  
    var escalaFinal = ESCALA * (escalaExtra || 1);  
    raiz.scaling.x *= escalaFinal;  
    raiz.scaling.y *= escalaFinal;  
    raiz.scaling.z *= escalaFinal;  
    raiz.position.x = raiz.position.x * ESCALA + x;  
    raiz.position.y = raiz.position.y * ESCALA + y;  
    raiz.position.z = raiz.position.z * ESCALA + z;  
    if (pai) raiz.setParent(pai);  
    return { raiz: raiz, animationGroups: res.animationGroups };  
}
```

Extra — código (passar a escala nos ciclos)

babylon final.ts

// no bloco 5 (obstaculosFixos), atualiza a chamada para passar o ultimo parametro (of.escala):

```
var objFixo = await carregarModelo(  
  "obstaculo" + i, of.ficheiro, of.x, of.y, 0, scene, nivel, of.escala  
);
```

// no bloco 6 (pontuaveis), atualiza tambem para passar pv.escala:

```
var raizPv = (await carregarModelo("pontuavel" + j, pv.ficheiro, pv.x, pv.y, 0, scene, nivel, pv.escala)  
).raiz;
```

Extra — adicionar o novo objeto aos obstáculos

- No teu projeto cria uma pasta modelos
- deves colocar os teus modelos no formato .glb nesta diretoria

babylon final.ts

```
// no array obstaculosFixos (bloco 5), ja podes adicionar o teu modelo
// com um caminho local (./modelos/...) e um parametro "escala" a gosto:
var obstaculosFixos = [
  { ficheiro: "./modelos/gothic_statue.glb", x: 9, y: 0, escala: 0.4 },
  { ficheiro: "Pilar_Low_Poly.glb", x: 24, y: 0 },
  { ficheiro: "Barrel.glb", x: 54, y: 0 },
  { ficheiro: "Barrel.glb", x: 70, y: 0 },
  { ficheiro: "Barrel.glb", x: 90, y: 0 },
];
```

- O que vais ver: O Pilar desaparece e no seu lugar (x: 9) surge a tua incrível estátua Gótica reduzida a 5% do seu tamanho original!

Pronto para ver o resultado?

Clica no fundo do teu VS Code em Go Live e vê a magia a acontecer.

Último desafio

És capaz de mudar a tua personagem por outro objeto à tua escolha?